

REVERSE ENGINEERING // MASTERY COURSE

PHASE 2

Tools

Ghidra, x64dbg aur helper tools — .exe ki ghadi kholne ke aujaar.

```
re@learning: ~/phase-2
$ ./start_learning.sh --phase 2
# Hinglish mein, simple examples ke saath – ek-ek step
$ status: READY ✓
```

 Detailed Theory

 Code + Line-by-line

 Interview Q&A

IN Hinglish

Reverse Engineering Mastery 3A • Self-Study Roadmap 8 • Phase 2 Handbook 93

Phase 2 — Tools Se Dosti (Aujaar Seekho)

Goal: Wo tools seekhna jinse hum `.exe` ki "ghadi" kholte hain. Ab tak theory thi, ab practical tools. Time: **2-3 hafte**.

2.0 — Tools Kyun Zaroori?

Assembly padhna aa gaya (Phase 1), lekin `.exe` ko assembly mein badalta kaun hai? **Tools**. Ye tools raw machine code ko humein padhne layak (assembly / C-jaisa) code mein dikhate hain, program ko chalate-rokte hain, aur memory dikhate hain.

Tools 3 category ke hote hain: 1. **Disassembler / Decompiler** — `.exe` → assembly / C-jaisa code (static) 2. **Debugger** — program ko chalate hue rok-rok ke dekhna (dynamic) 3. **Helper tools** — file inspect karna, strings nikaalna, hex dekhna

2.1 — Disassembler vs Decompiler (Farak Samjho)

	Disassembler	Decompiler
Output	Assembly code	C-jaisa high-level code
Padhna	thoda mushkil	aasan
Accuracy	100% sahi	approximate (kabhi galat)
Example	IDA, objdump	Ghidra decompiler, Hex-Rays

- **Disassembler:** machine code → assembly. Ye exact hota hai.
- **Decompiler:** assembly → C-jaisa code. Ye padhna aasan hai, but variable naam ajeeb (`local_8` , `uVar1`) aur kabhi galti bhi karta hai.

Best strategy: Dono use karo. Decompiler se overall logic samjho, aur jahan doubt ho wahan assembly (disassembler) dekho.

2.2 — Main Tools (Comparison)

Tool	Kya Hai	Cost	Beginner?
Ghidra ★	NSA ka banaya, free, powerful. Disassembler + decompiler dono	Free	✓ Best se shuru
IDA Free	Industry standard, free version (64-bit disassembler)	Free	✓
x64dbg	Windows debugger (runtime analysis)	Free	✓
Cutter / Rizin	Open source, modern UI	Free	✓
Binary Ninja	Achha UI, powerful	Paid	⚠ baad mein
GDB	Linux ka classic debugger	Free	Linux ke liye

Recommendation: Ghidra + x64dbg se shuru karo. Dono free hain. Ghidra static analysis ke liye, x64dbg dynamic ke liye. Yahi combo 90% kaam kar dega.

2.3 — Ghidra (Sabse Zaroori Tool)

Ghidra free hai aur **decompiler** bhi deta hai (assembly ko C-jaisa dikhata hai) — isiliye beginners ke liye best.

Ghidra Setup

1. Java (JDK 17+) install karo (Ghidra ko chahiye).
2. Ghidra download karo (official NSA GitHub se).
3. Run karo, ek **Project** banao.
4. Apna `.exe` import karo (File → Import File).
5. Analysis chalao ("Analyze" — Yes bolo, defaults theek hain).

Ghidra ke main windows (parts)

- **Listing** (beech mein) — assembly code, address ke saath.
- **Decompiler** (right side) — C-jaisa code. ★ Sabse zyada yahi padhoge.
- **Symbol Tree** (left) — functions, imports, exports ki list.
- **Program Trees / Data Type Manager** — structure aur types.

Ghidra mein basic kaam

- **Function dhoondo:** Symbol Tree → Functions. `main` ya `entry` dhoondo.

- **Rename karo:** Kisi variable/function pe click → `L` dabao → naya naam do (jaise `local_8` ko `user_input`). Isse code padhna aasan hota hai.
- **Cross-references (XREF):** Kisi function pe right-click → "References" → dekho use kahan-kahan call hua. ★ Bahut useful.
- **Comments:** `;` dabao ke apne notes likho.

Tip: Ghidra mein rename aur comment karte raho jaise-jaise samajhte ho. Ye "map banane" jaisa hai — jitna map banega, utna aasan.

2.4 — x64dbg (Windows Debugger)

x64dbg program ko **chalate hue** rok-rok ke dekhne ke liye hai (dynamic analysis — Phase 4 mein detail).

x64dbg ke main parts

- **CPU window** — assembly instructions (abhi kaunsi chal rahi).
- **Registers** (right) — live values (eax, ebx, eip...).
- **Dump** (niche) — memory ka hex view.
- **Stack** (right-niche) — stack ki live values.

Basic controls

Key	Kaam
<code>F2</code>	Breakpoint lagao/hatao (yahan ruk jaao)
<code>F7</code>	Step Into (function ke andar jao)
<code>F8</code>	Step Over (function ko chala do, andar mat jao)
<code>F9</code>	Run (agle breakpoint tak chalao)

Abhi in tools ko sirf install karke ghoomo. Deep use Phase 3-4 mein.

2.5 — Helper Tools (Chhote But Kaam Ke)

Ye chhote tools RE workflow mein bahut kaam aate hain:

Tool	Kaam
Detect It Easy (DIE) ★	File kaunse language/compiler se bani, packed hai ya nahi — instant batata hai
PE-bear / PExview / CFF Explorer	<code>.exe</code> (PE file) ki internal structure dekhna — headers, sections, imports
Strings (Sysinternals)	File mein chhupe readable text nikaalna (URLs, passwords, messages)
HxD (Hex Editor)	File ko raw bytes mein dekhna/edit karna
Dependency Walker / PE imports	Program kaunsi DLL/functions use karta hai
Process Monitor / Process Explorer	Chalte hue program ki activity (files, registry, network)

PE File Structure (Windows .exe) — Basic

Windows executables "PE format" mein hote hain. Iske parts: - **DOS Header** — purana compatibility header (`MZ` se shuru). - **PE Header** — asli metadata (32/64 bit, entry point address). - **Sections:** - `.text` — actual code (instructions) - `.data` — initialized data (variables jinki value pehle se hai) - `.rdata` — read-only data (strings, constants) - `.idata` — imports (kaunse DLL/functions use ho rahe) - `.rsrc` — resources (icons, images)

`.text` mein code hota hai (yahan reverse karte hain), `.rdata` mein strings (yahan clues milte hain). Linux mein iska equivalent "ELF format" hota hai.

2.6 — Pehla Practical (Zaroor Karo)

1. Ek simple C program khud likho (jaise password check karne wala).
2. Compile karke `.exe` banao.
3. **DIE** mein kholo — dekho kya batata hai (compiler, bits).
4. **Ghidra** mein import karo, analyze karo.
5. `main` function dhoondo, decompiler mein padho — apna hi likha logic pehchano!
6. **x64dbg** mein kholo, `main` pe breakpoint lagao, `F8` se step karo.

Apna hi program reverse karna best starting point hai — kyunki tumhe pehle se pata hai andar kya hai, toh tools samajhna aasan.

Key Takeaways (Ek Line Mein)

1. **Disassembler** = assembly (exact); **Decompiler** = C-jaisa code (aasan but approximate). Dono use karo.
2. **Ghidra** (free, decompiler ke saath) = best beginner tool for static analysis.
3. **x64dbg** = best free Windows debugger for dynamic analysis.
4. Ghidra mein **rename** aur **XREF** (cross-references) sabse useful features.
5. **DIE** se file identify karo (packed? kaunsa compiler?).
6. Windows `.exe` = **PE format**; code `.text` section mein, strings `.rdata` mein.
7. Debugger keys: `F2` breakpoint, `F7` step into, `F8` step over, `F9` run.

✅ **Phase 2 Complete Jab:** Tum Ghidra aur x64dbg install karke ek program khol sako, functions dhoond sako, decompiler padh sako, aur basic navigate kar sako. Ab asli analysis shuru (Phase 3)!



Phase 2 — Interview Questions & Answers (Hinglish)

RE Tools se related important interview questions, simple Hinglish jawaab ke saath.

Q1. Disassembler aur Decompiler mein kya farak hai?

Ans: **Disassembler** machine code ko assembly mein badalta hai — ye 100% exact hota hai (jaise IDA, objdump). **Decompiler** us code ko C-jaise high-level code mein badalta hai — padhna aasan hota hai but approximate hota hai, variable naam ajeeb aur kabhi galti bhi (jaise Ghidra decompiler, Hex-Rays). Best strategy: dono use karo.

Q2. Ghidra kya hai aur beginners ke liye kyun accha hai?

Ans: Ghidra ek free, powerful RE tool hai jise NSA ne banaya. Ye disassembler aur decompiler dono deta hai — matlab assembly ke saath C-jaisa code bhi dikhata hai, jo padhna aasan hai. Free hone aur decompiler dene ki wajah se beginners ke liye best hai.

Q3. Debugger kya hota hai aur wo disassembler se kaise alag hai?

Ans: **Disassembler** program ko chalay bina uska code dikhata hai (static analysis). **Debugger** program ko **chalate hue** rok-rok ke dekhne deta hai — breakpoints lagana, ek-ek line chalana, live register aur memory values dekhna (dynamic analysis). Example: x64dbg, GDB.

Q4. Static analysis aur dynamic analysis mein kya farak hai?

Ans: **Static analysis** program ko chalay bina, sirf code padh ke samajhna hai (Ghidra se). **Dynamic analysis** program ko chalate hue, runtime pe values dekhkar samajhna hai (x64dbg se). Dono complementary hain — often dono milaake use karte hain.

Q5. Cross-reference (XREF) kya hota hai aur kyun useful hai?

Ans: XREF batata hai ki koi function ya variable code mein **kahan-kahan use** hua hai. Jaise agar tumhe "password" string mili, toh uske XREF se pata chalega ki wo string kaunse function mein use ho rahi — seedha important logic tak pahunch jaate ho. RE mein clues follow karne ka ye best tarika hai.

Q6. PE file format kya hai?

Ans: PE (Portable Executable) Windows executables (`.exe` , `.dll`) ka format hai. Ismein DOS header, PE header (32/64-bit info, entry point), aur sections hote hain jaise `.text` (code), `.data` (variables), `.rdata` (strings/constants), `.idata` (imports), `.rsrc` (resources). Linux ka equivalent ELF format hai.

Q7. `.text` aur `.rdata` sections mein kya hota hai?

Ans: `.text` section mein program ka actual code (instructions) hota hai — yahin reverse karte hain. `.rdata` section mein read-only data hota hai jaise strings aur constants — yahan often passwords, URLs, error messages jaise clues milte hain.

Q8. "Strings" tool kya karta hai aur RE mein kyun kaam aata hai?

Ans: Strings tool file ke andar chhupe readable text (printable characters) nikaal deta hai. RE mein ye pehla step hota hai — often hardcoded passwords, URLs, API keys, error messages, ya hints seedha strings mein mil jaate hain, bina deep analysis ke.

Q9. Packed binary kya hota hai aur use kaise detect karte hain?

Ans: Packed binary wo hai jiska code compress ya encrypt kar diya gaya hai (jaise UPX packer se), taaki analysis mushkil ho. Aisi file mein strings kam dikhte hain aur code samajh nahi aata. Ise **Detect It Easy (DIE)** jaise tool se detect karte hain — wo batata hai file packed hai ya nahi aur kaunse packer se.

Q10. Ghidra mein rename karna kyun important hai?

Ans: Decompiler variables ko ajeeb naam deta hai jaise `local_8`, `uVar1`. Jaise-jaise hum samajhte hain ki kaunsa variable kya kar raha, unhe meaningful naam de dete hain (jaise `user_input`, `password_check`). Isse code padhna aur logic samajhna bahut aasan ho jaata hai — ye "map banane" jaisa hai.

Q11. x64dbg ke basic controls (F7, F8, F9) kya karte hain?

Ans: - **F2** — breakpoint lagana/hatana (program yahan ruk jaayega). - **F7** — Step Into (function ke andar chale jaao). - **F8** — Step Over (function ko chala do, but andar mat jaao). - **F9** — Run (agle breakpoint tak chalao).

Q12. Kaunse tools se RE shuru karna best hai aur kyun?

Ans: **Ghidra + x64dbg** — dono free hain aur milkar 90% kaam kar dete hain. Ghidra static analysis (code padhna, decompiler) ke liye aur x64dbg dynamic analysis (runtime debugging) ke liye. Baad mein IDA, Binary Ninja jaise advanced tools try kar sakte ho.

Phase 2 — Interview Questions & Answers (English)

The same RE Tools interview questions, answered in clear English for formal interviews.

Q1. What is the difference between a disassembler and a decompiler?

Ans: A **disassembler** converts machine code into assembly — it's 100% exact (e.g., IDA, objdump). A **decompiler** converts that code into C-like high-level code — easier to read but approximate, with odd variable names and occasional mistakes (e.g., Ghidra's decompiler, Hex-Rays). Best practice is to use both.

Q2. What is Ghidra and why is it good for beginners?

Ans: Ghidra is a free, powerful RE tool developed by the NSA. It provides both a disassembler and a decompiler — showing assembly alongside readable C-like code. Being free and including a decompiler makes it ideal for beginners.

Q3. What is a debugger and how does it differ from a disassembler?

Ans: A **disassembler** shows a program's code without running it (static analysis). A **debugger** lets you inspect a program while it **runs** — setting breakpoints, stepping through instructions, and viewing live register and memory values (dynamic analysis). Examples: x64dbg, GDB.

Q4. What is the difference between static and dynamic analysis?

Ans: **Static analysis** means understanding a program by reading its code without running it (using Ghidra). **Dynamic analysis** means understanding it while running it and observing runtime values (using x64dbg). They're complementary and often used together.

Q5. What is a cross-reference (XREF) and why is it useful?

Ans: An XREF tells you where a function or variable is **used** in the code. For example, if you find a "password" string, its XREFs reveal which function uses it — leading you straight to the important logic. It's the best way to follow clues in RE.

Q6. What is the PE file format?

Ans: PE (Portable Executable) is the format for Windows executables (`.exe` , `.dll`). It contains a DOS header, a PE header (32/64-bit info, entry point), and sections such as `.text` (code), `.data` (variables), `.rdata` (strings/constants), `.idata` (imports), and `.rsrc` (resources). The Linux equivalent is the ELF format.

Q7. What is stored in the `.text` and `.rdata` sections?

Ans: The `.text` section holds the program's actual code (instructions) — where reversing happens. The `.rdata` section holds read-only data such as strings and constants — often revealing clues like passwords, URLs, and error messages.

Q8. What does the "Strings" tool do and why is it useful in RE?

Ans: The Strings tool extracts readable text (printable characters) from a file. In RE it's often the first step — hardcoded passwords, URLs, API keys, error messages, or hints frequently appear directly in the strings, without deep analysis.

Q9. What is a packed binary and how do you detect it?

Ans: A packed binary has its code compressed or encrypted (e.g., by UPX) to make analysis harder. Such files show few readable strings and unclear code. You detect them with a tool like **Detect It Easy (DIE)**, which reports whether a file is packed and by which packer.

Q10. Why is renaming important in Ghidra?

Ans: The decompiler gives variables generic names like `local_8`, `uVar1`. As you understand what each does, you rename them to meaningful names (like `user_input`, `password_check`). This makes the code far easier to read and reason about — it's like drawing a map as you explore.

Q11. What do the basic x64dbg controls (F7, F8, F9) do?

Ans: - **F2** — set/remove a breakpoint (the program pauses here). - **F7** — Step Into (enter the called function). - **F8** — Step Over (execute the function without entering it). - **F9** — Run (continue to the next breakpoint).

Q12. Which tools are best to start RE with and why?

Ans: Ghidra + x64dbg — both are free and together handle about 90% of the work. Ghidra for static analysis (reading code, decompiler) and x64dbg for dynamic analysis (runtime debugging). Later you can try advanced tools like IDA or Binary Ninja.